PROJECT SPECIFICATION

RK3566
图形界面系统

项目说明书 · Project Specification

目　录

1. 项目概述 1

2. 系统架构 1

3. 编译与部署 2

3.1 编译环境 2

3.2 编译命令示例 2

3.3 资源文件 3

3.4 运行环境变量 3

3.5 运行 4

4. 用户界面总览 4

5. 应用模块说明 5

5.1 相册（Album） 5

5.2 画板（Paint） 5

5.3 弹球（Bounce） 6

5.4 贪吃蛇（Snake） 6

5.5 五子棋（Gomoku） 6

5.6 系统工具 7

6. 开发接口说明（供二次开发） 7

6.1 绘制原语（ui.h） 7

6.2 输入事件（input.h） 8

6.3 动画引擎（animation.h） 9

6.4 BMP 工具（bmp_display.h） 10

7. 配置与环境变量 10

7.1 运行时环境变量 10

7.2 构建时配置 11

8. 故障排除 11

8.1 无法打开 Framebuffer 11

8.2 触摸无响应 12

8.3 资源文件未找到 12

1. 项目概述

RK3566 GUI System 是一个基于 Linux Framebuffer 的轻量级嵌入式图形用户界面系统，专为 RK3566 平台设计。系统集成了触摸交互、多窗口应用、游戏和系统工具，所有界面均通过 /dev/fb0 直接绘制，无需 X11/Wayland，资源占用极低。

本系统面向嵌入式开发场景，提供从底层帧缓冲绘制到上层应用框架的完整解决方案。开发者可基于本系统快速构建触摸式图形应用，亦可作为二次开发的参考实现，深入理解 Linux Framebuffer 编程模型与触摸事件处理机制。

- **硬件平台**：RK3566（ARM Cortex-A55）
- **显示**：支持 32bpp Framebuffer（/dev/fb0），自动适配屏幕分辨率
- **输入**：触摸屏（/dev/input/eventX），支持点击、滑动手势，以及物理按键（返回、电源）
- **开发语言**：C（ANSI C99）
- **依赖**：Linux 标准库（libc）、pthread、linux/fb.h

2. 系统架构

系统由以下核心模块组成，各模块职责清晰、相互解耦，便于独立维护与功能扩展。底层绘制模块封装 Framebuffer 操作，输入处理模块统一管理触摸与按键事件，应用框架提供通用 UI 组件，上层应用基于这些基础设施构建具体功能。

表 2-1　系统核心模块一览

模块	文件	功能
底层绘制	ui.c	Framebuffer 映射、像素/矩形/圆/线/文字绘制、BMP 图片显示
输入处理	input.c	触摸屏事件解析（按下/移动/抬起）、手势识别（滑动、点击）、物理按键
应用框架	app_ui.c	通用 UI 组件（卡片、图标、箭头）
系统启动	main.c	初始化、开机动画、登录、主界面循环

模块	文件	功能
启动动画	boot_anim.c	开机/关机动画（BMP 淡入/保持）
动画引擎	animation.c	序列帧 BMP 播放（循环/单次）
登录管理	login.c	虚拟键盘输入、账号密码验证、密码修改
主界面	main_ui.c	主页网格布局，显示应用卡片和系统工具
相册	album.c	浏览目录下的 BMP 图片，支持缩略图网格和单图查看
画板	paint.c	调色板、笔刷粗细、撤销/清空、触摸绘画
弹球	ball.c	弹球游戏（Bounce），带难度选择和分数记录
贪吃蛇	snake.c	经典贪吃蛇，可调节速度步进
五子棋	gomoku.c	双人五子棋，带悔棋和重新开局
游戏菜单	game_menu.c	统一的游戏入口菜单（难度、设置、排行榜）
游戏 UI	game_ui.c	游戏公共 UI（侧边栏、计分板、D-Pad 虚拟按键）
触控按键	touch_ctrl.c	半透明虚拟方向键和动作按钮（用于无实体按键设备）
BMP 工具	bmp_display.c / bmp_util.c	BMP 文件解析、缩放（最近邻）

3. 编译与部署

3.1 编译环境

本系统采用交叉编译方式构建，需准备支持 ARM 架构的交叉编译工具链。推荐使用 arm-linux-gnueabihf-（32 位 ARM）或 aarch64-linux-gnu-（64 位 ARM）工具链，并确保工具链支持 C99 标准与 pthread 线程库链接。

- 交叉编译工具链（arm-linux-gnueabihf- 或 aarch64-linux-gnu-）
- 支持 -std=c99、-pthread 链接

3.2 编译命令示例

将所有 .c 源文件编译为可执行文件 rk_gui，需在命令行中显式列出全部依赖源文件。若仅需单独模块，可按需链接对应源文件。

```bash
# 将所有 .c 文件编译为可执行文件（需包含所有依赖）
arm-linux-gnueabihf-gcc -std=c99 -O2 -pthread \
    main.c ui.c input.c app_ui.c main_ui.c login.c \
    boot_anim.c animation.c album.c paint.c ball.c \
    snake.c gomoku.c game_menu.c game_ui.c \
    touch_ctrl.c bmp_display.c bmp_util.c \
    -o rk_gui
```

若需要单独模块，可按需链接。

3.3 资源文件

系统运行需要以下资源目录结构，可通过环境变量 RK_GUI_RES 指定根目录，默认为当前工作目录下的 ./res。资源目录包含开机动画、关机动画与相册图片三个子目录，所有图片须为 24 位 BMP 格式。

```text
res/
├── boot/        # 开机动画图片（单张 bmp，推荐 24 位）
│   └── frame000.bmp   （或任意 .bmp，自动选取第一张）
├── shutdown/    # 关机动画图片（单张 bmp）
│   └── frame000.bmp
└── album/       # 相册图片目录（存放 .bmp 文件）
    └── (任意 bmp 文件)
```

开机动画 播放时以白屏为背景，图片从全白淡入到全彩，持续约 4 秒。触摸任意位置可跳过。

关机动画 全屏显示关机图片，保持 5 秒后退出。

3.4 运行环境变量

系统支持通过环境变量灵活配置运行时参数，包括 Framebuffer 设备路径、触摸

输入设备路径、资源根目录及登录验证开关。下表列出全部支持的环境变量及其默认值。

表 3-1　运行环境变量

变量	说明	默认值
RK_GUI_FB	Framebuffer 设备路径	/dev/fb0
RK_GUI_INPUT	触摸输入设备路径	/dev/input/event6
RK_GUI_RES	资源根目录	./res
RK_SKIP_LOGIN	设为 1 跳过登录验证（调试用）	未设置

3.5 运行

编译完成后，需确保运行用户对 Framebuffer 设备节点与触摸输入设备节点具备读写权限。推荐使用 sudo 提权运行，或通过 udev 规则为指定用户授予设备访问权限。

```bash
# 确保设备节点可读写
sudo ./rk_gui
```

4. 用户界面总览

系统启动后依次经历三个阶段：开机动画、登录界面与主界面。开机动画以淡入方式呈现启动画面，登录界面接收用户名与密码输入，主界面采用卡片式布局组织应用与系统工具。各阶段衔接流畅，整体交互逻辑符合嵌入式触摸设备的使用习惯。

1. **开机动画** — 淡入显示启动画面。
2. **登录界面** — 输入用户名和密码（默认均为 123）。密码错误 3 次后自动放行（演示模式）。
3. **主界面** — 卡片式布局，分为“应用区”（5 个大卡片）和“工具区”（3 个小卡

片）。

主界面支持以下交互操作：

- **点击**卡片启动对应应用。
- **上下滑动**移动焦点（选中高亮）。
- **状态栏**显示当前时间和日期。

5. 应用模块说明

本章详细介绍系统内置的各应用模块，包括相册、画板、弹球、贪吃蛇、五子棋及系统工具。每个模块均说明其功能定位、操作方式与交互细节，便于使用者快速上手，亦便于开发者理解模块实现要点。

5.1 相册（Album）

- **功能**：浏览 res/album/ 目录下所有 .bmp 图片（大小写不敏感）。
- **操作**：
  - 缩略图网格（4 列）：点击缩略图进入单图查看。
  - 单图模式：点击图片切换 1×/2× 缩放；左右滑动切换上一张/下一张；上/下滑也切换缩放。
  - 右上角 [] 按钮返回缩略图网格。
  - 左上角 < 按钮返回主界面。
- **提示**：底部栏显示操作提示。

5.2 画板（Paint）

- **功能**：自由绘画，支持颜色选择、笔刷粗细、撤销、清空。
- **操作**：
  - 顶部调色板：点击选择颜色（8 种预定义色）。
  - 底部工具栏：S/M/L 选择笔刷粗细（2/5/10 像素）。
  - Undo 撤销上一笔（最多保存 30 笔）。
  - Clear 清空画布。

  - Back 返回主界面。
- **绘制**：在画布区域触摸拖拽绘制；单击也可落点。

5.3 弹球（Bounce）

- **玩法**：控制底部挡板反弹小球，接住得分，漏掉则游戏结束。
- **操作**：
  - 触摸并拖动挡板左右移动（在游戏区域拖拽）。
  - 也可点击 D-Pad 虚拟按键（左侧）控制挡板移动。
  - 右侧面板：PAUSE 暂停/继续，BACK 返回菜单。
- **难度**：在游戏菜单中可选择 EASY/NORMAL/HARD，影响球速和挡板宽度。
- **计分**：每接住一次得 10 分，连续得分后球速略有提升，分数越高速度越快。
- **排行榜**：记录前 10 名最高分（按分数降序）。

5.4 贪吃蛇（Snake）

- **玩法**：控制蛇头移动吃掉食物，避免撞墙或撞自身。
- **操作**：
  - 滑动屏幕方向（上/下/左/右）控制蛇头方向。
  - 也可点击 D-Pad 虚拟按键。
  - 右侧面板：PAUSE 暂停/继续，BACK 返回菜单。
- **设置**：游戏菜单中可调节 SPEED STEP（1~10），数值越高每吃到一个食物后加速越快（步进值）。
- **计分**：每吃一个食物得 10 分，蛇身变长，速度逐渐增加至下限 55ms/步。

5.5 五子棋（Gomoku）

- **玩法**：双人在 15×15 棋盘上轮流落子，先连成五子者胜。
- **操作**：
  - 点击棋盘交叉点落子（黑先白后）。
  - 底部按钮：UNDO 悔棋（撤回上一步），NEW ROUND 重新开局，BACK 返回菜单。
- **状态**：顶部显示当前回合（黑/白）或胜者（黑/白胜/平局）。

- **交互**：点击棋盘外区域不会落子。

5.6 系统工具

- **Security（设置密码）**：验证当前密码后，输入并确认新密码，修改内存中的密码（演示用，重启后重置）。
- **About（关于）**：显示系统名称、构建日期、应用列表。
- **Power（关机）**：播放关机动画后退出系统。

6. 开发接口说明（供二次开发）

本章面向二次开发者，介绍系统对外暴露的核心 API，包括底层绘制原语、输入事件结构、动画引擎接口与 BMP 工具函数。开发者可基于这些接口快速构建自定义应用，无需重新实现底层逻辑。

6.1 绘制原语（ui.h）

ui.h 提供完整的 Framebuffer 绘制原语集合，涵盖基本图元（像素、矩形、圆、线）、复合图元（圆角矩形）、文字渲染与 BMP 图片显示。所有绘制函数均自动处理屏幕裁剪，开发者无需关心越界问题。

表 6-1　绘制原语函数列表

函数	说明
ui_init()	打开 fb0 并 mmap，获取屏幕尺寸
ui_close()	释放 mmap 并关闭 fd
ui_clear(color)	全屏填充颜色
ui_put_pixel(x, y, color)	画单像素
ui_fill_rect(x, y, w, h, color)	填充矩形（自动裁剪）
ui_draw_rect(...)	矩形边框
ui_fill_circle(cx, cy, r, color)	填充圆
ui_draw_circle(...)	圆边框
ui_draw_line(x0, y0, x1, y1, color)	Bresenham 直线

函数	说明
ui_draw_hline / vline	水平 / 垂直线
ui_fill_round_rect(...)	圆角矩形填充
ui_draw_round_rect(...)	圆角矩形边框
ui_draw_text(x, y, str, fg, bg, scale)	绘制 8×16 点阵 ASCII 字符（支持缩放）
ui_draw_text_center(...)	文字水平居中
ui_draw_bmp(path, x, y, scale)	绘制 24 位 BMP（自动缩放）
ui_button_hit()	检测点击是否在按钮区域内

> **颜色格式** 0xAARRGGBB，支持透明度（在某些绘图操作中可使用透明混合）。

6.2 输入事件（input.h）

input.h 定义了输入事件的类型枚举与事件结构体，并提供初始化、轮询与阻塞等待三类接口。触摸坐标自动映射到屏幕像素范围，滑动阈值约为屏幕短边的 1/10。

```c
typedef enum {
    INPUT_NONE,
    INPUT_TAP,            // 点击（按下+抬起，位移很小）
    INPUT_SWIPE_UP/DOWN/LEFT/RIGHT,
    INPUT_TOUCH_DOWN,
    INPUT_TOUCH_MOVE,
    INPUT_TOUCH_UP,
    INPUT_KEY_BACK,
    INPUT_KEY_POWER,
    INPUT_QUIT,           // 接收到退出信号
} input_event_type_t;

typedef struct {
    input_event_type_t type;
    int x, y;             // 屏幕坐标（已映射）
} input_event_t;

int input_init(dev_path, screen_w, screen_h);
int input_poll(input_event_t *out);         // 非阻塞，返回手势/点击事件
int input_poll_raw(input_event_t *out);     // 返回原始触摸事件
（DOWN/MOVE/UP）
int input_wait_event(input_event_t *out, int timeout_ms);  // 阻塞等待
```

触摸坐标自动映射到屏幕像素范围，滑动阈值约为屏幕短边的 1/10。

6.3 动画引擎（animation.h）

animation.h 提供序列帧 BMP 播放能力，支持循环与单次模式，可指定帧间隔、是否填充背景及背景颜色。当 last_idx 设为 -1 时，引擎自动扫描目录下最大编号的帧文件。

```c
typedef struct {
    const char *dir;        // 图片目录
    const char *prefix;     // 文件名前缀（默认 "frame"）
    const char *suffix;     // 后缀（默认 ".bmp"）
    int first_idx, last_idx; // 若 last_idx=-1 则自动扫描最大编号
    int frame_ms;           // 帧间隔（ms）
    int loop;               // 是否循环
    int fill_bg;            // 是否每帧填充背景
    unsigned int bg_color;
} anim_opts_t;

int animation_play(const anim_opts_t *opts, volatile int *interrupt);
int animation_scan(anim_opts_t *opts);   // 自动探测 last_idx
```

6.4 BMP 工具（bmp_display.h）

bmp_display.h 提供 BMP 文件解析与缩放工具函数，支持读取 BMP 头部信息、解包像素数据及最近邻缩放。所有函数均返回解包后的 RGB 数据，便于上层绘制函数直接使用。

- `read_bmp_info(path, bmp_info_t*)` — 读取 BMP 头部信息（宽、高、位深、行大小、是否自底向上）。
- `read_bmp_pixels(path, info*)` — 返回解包后的 RGB 数据（顺序 BGR，每像素 3 字节）。
- `scale_bmp_nearest(src, w, h, row_size, scale, out_w, out_h)` — 最近邻缩放，返回新的 RGB 数据。

7. 配置与环境变量

本章汇总系统的运行时与构建时配置项。运行时配置通过环境变量传入，构建时配置则通过修改源码头文件中的宏定义实现。合理调整这些配置项可适配不同硬件平台与使用场景。

7.1 运行时环境变量

运行时环境变量允许在不重新编译的情况下调整系统行为，适用于不同设备节点路径或调试场景。下表与第 3.4 节内容一致，此处再次列出以便快速查阅。

表 7-1 运行时环境变量

变量	说明	默认值
RK_GUI_FB	Framebuffer 设备	/dev/fb0
RK_GUI_INPUT	触摸输入设备	/dev/input/event6
RK_GUI_RES	资源根目录	./res
RK_SKIP_LOGIN	跳过登录（任意非空值）	未设置

7.2 构建时配置

构建时配置通过修改源码头文件中的宏定义实现，需重新编译生效。颜色定义集中在 app_ui.h 中，游戏参数则分散在各自的游戏源文件内。

- 颜色定义可在 `app_ui.h` 中调整（`APP_COLOR_*` 宏）。
- 游戏参数（速度、挡板宽度、球速等）在 `ball.c / snake.c` 内部可微调。

8. 故障排除

本章列举系统运行中常见的三类故障及其排查方法，涵盖 Framebuffer 设备访问、触摸输入响应与资源文件加载。遇到问题时，建议按本章顺序逐项排查，多数问题可通过权限调整或环境变量配置解决。

8.1 无法打开 Framebuffer

Framebuffer 设备无法打开通常由权限不足或设备路径配置错误引起。请按以下步骤逐项排查，确认运行用户对设备节点具备读写权限，并检查环境变量配置是否正确。

- 确保运行用户有读写 /dev/fb0 权限（sudo 或设置 udev 规则）。
- 检查环境变量 RK_GUI_FB 是否正确。
- 确认屏幕支持 32bpp（fbset 查看）。

8.2 触摸无响应

触摸无响应多由输入设备路径错误或触摸 IC 上报范围与屏幕分辨率不匹配引起。系统会自动完成坐标映射，但前提是能够正确读取 ABS_X/Y 范围。

- 检查 RK_GUI_INPUT 指向正确的 event 设备（可用 evtest 测试）。
- 若坐标映射错误，检查触摸 IC 上报范围是否与屏幕分辨率匹配（系统会自动映射，但需正确读取 ABS_X/Y 范围）。

8.3 资源文件未找到

资源文件未找到通常由 RK_GUI_RES 路径配置错误或图片格式不符合要求引起。系统对 BMP 文件格式有严格校验，非 24 位 BMP 将触发错误。

- 确认 RK_GUI_RES 指向包含 boot/、shutdown/、album/ 的目录。
- 图片必须为 24 位 BMP，否则 ui_draw_bmp 会报错（代码中有检查）。